

Informe Técnico de Observaciones — API Integra ESAVI

Proyecto	API Integra ESAVI — <code>api-integra-esavi</code> rama <code>main</code>
Fecha	14 de junio de 2026
Elaborado por	Equipo de Desarrollo
Alcance	<code>src/</code> · <code>upload_files/</code> · <code>docker-compose.yaml</code>

Informe de observaciones técnicas identificadas mediante análisis estático del backend. Cada observación incluye referencia al archivo y línea de código correspondiente.

OBS-01 · Gestión y carga de catálogos

Archivos principales: `upload_files/` · `src/integrator/service/seed.service.ts`

Formatos mixtos con archivo duplicado. La carpeta `upload_files/` almacena catálogos sin criterio de formato unificado: cuatro archivos CSV y cuatro archivos Excel `.xlsx`. El catálogo de homologación de vacunas WHODrug-VigiFlow existe simultáneamente en ambos formatos con contenido divergente (CSV: 34 registros, Excel: 35 registros). El servicio solo carga la versión Excel (`loadWhodrugHomologacionVfFromExcel`, línea 1423); el CSV existe en el repositorio pero nunca es leído, por lo que actualizarlo no tiene ningún efecto. Adicionalmente, los dos archivos Excel de WHODrug tienen fecha incrustada en el nombre (`20260126-...`), acoplando las rutas del código fuente (líneas 1321 y 1424 de `seed.service.ts`) al nombre del archivo; cada reemplazo del archivo obliga a modificar el código.

Carga completa en cada arranque del servidor. `SeedService` implementa `OnApplicationBootstrap` (línea 29), ejecutando `seedData()` automáticamente en cada reinicio de la aplicación. El proceso carga ~13,436 registros mediante un bucle que emite un `findOne()` (SELECT) y condicionalmente un `save()` (INSERT) por cada fila, sin uso de bulk insert, generando **más de 26,000 queries individuales por inicio**. Los rangos de lectura de los archivos Excel están hardcodedados en el código (`'A3:I111195'` línea 1194, `'A2:T318'` línea 1324); si el archivo fuente crece más allá del límite definido, los registros adicionales no se cargan y el proceso finaliza sin error ni advertencia alguna.

TC_GRUPO_ETARIO tratada como tabla transaccional. El método `cleanData()` (línea 170) incluye `TRUNCATE TABLE "dhi_esavi"."TC_GRUPO_ETARIO"` dentro del bloque que limpia tablas `TR_*`. Esta es una tabla de catálogo (prefijo `TC_`), no transaccional, y es eliminada y reconstruida en cada arranque del servidor junto con los datos de pacientes.

Siete tablas `TC_` sin patrón de carga unificado. Cada tabla de catálogo tiene fuente y mecanismo de carga distinto (hardcoded, CSV o Excel) sin criterio documentado. La tabla `TC_CATALOGO` recibe datos de provincias desde dos fuentes activas simultáneamente: 32 entradas hardcodedadas en

`seedCatalogos()` (líneas 358–948) y 24 registros desde `provincias_ecuador.csv` en `loadProvinciasFromCSV()`. El objeto de auditoría `IAuditoria` se instancia por separado en los diez métodos de carga con valores inconsistentes entre sí (`updatedBy: ''` en provincias, `updatedBy: 'System'` en parroquias); los comentarios `TODO` en líneas 1204 y 1274 indican que la auditoría está incompleta. Los controladores `ct-icd10meddra.controller.ts`, `ct-symptom211t.controller.ts`, `whodrug-homologavacs.controller.ts` y `whodrug-vacstemp.controller.ts` están registrados en el módulo pero no definen ningún endpoint, dejando más de 11,260 registros de esas tablas inaccesibles por API. El repositorio contiene simultáneamente `package-lock.json` (npm) y `pnpm-lock.yaml` (pnpm), lo que puede producir árboles de dependencias distintos según el entorno.

OBS-02 · Seguridad

Archivos: `src/integrator/controller/seed.controller.ts` · `docker-compose.yaml` · `src/main.ts`

Operaciones destructivas sin autenticación. `SeedController` expone tres endpoints sin ningún guard de autenticación ni autorización:

Endpoint	Operación
<code>POST /v1/seed</code>	Recarga ~13,436 registros de catálogo en base de datos
<code>DELETE /v1/seed</code>	Ejecuta <code>TRUNCATE</code> sobre 9 tablas transaccionales
<code>DELETE /v1/seed/tr-tables</code>	Consulta <code>information_schema</code> y ejecuta <code>TRUNCATE</code> dinámico sobre todas las tablas <code>TR_*</code>

El endpoint `DELETE /v1/seed/tr-tables` es el más crítico: identifica en tiempo de ejecución todas las tablas con prefijo `TR_` y ejecuta `TRUNCATE ... CASCADE` sobre cada una (líneas 1493–1524 de `seed.service.ts`), incluyendo datos reales de pacientes y eventos adversos. Cualquier cliente HTTP con acceso a la URL puede invocar esta operación sin presentar credencial. El sistema define un esquema `X-API-KEY` visible en Swagger (`main.ts` líneas 43–51), pero no existe ningún guard implementado que lo valide. Keycloak está configurado en infraestructura (`docker-compose.yaml`) pero no está integrado en la aplicación NestJS.

Credenciales en texto plano en el repositorio. `docker-compose.yaml` contiene credenciales de acceso commiteadas directamente:

```
POSTGRES_PASSWORD: dhis
KC_DB_PASSWORD: password123
KC_BOOTSTRAP_ADMIN_PASSWORD: admin_password
```

Estas credenciales quedan registradas en el historial de Git y son accesibles para cualquier persona con acceso de lectura al repositorio. La credencial del administrador de Keycloak otorga control total sobre el servidor de identidad del sistema.

CORS permisivo por defecto. En `main.ts` (línea 17), cuando la variable `CORS_ORIGINS` no está definida, se asigna `origin: true`, permitiendo solicitudes desde cualquier dominio sin restricción. En `.env.example` la variable aparece vacía (`CORS_ORIGINS=`), activando este comportamiento permisivo en

todo entorno donde no se configure explícitamente. La API procesa datos de salud sensibles de pacientes.

OBS-03 · Calidad de código

Archivos: `src/integrator/service/whodrug-vacstemp.service.ts` · `src/integrator/integrator.module.ts` · `src/integrator/service/catalogo.service.ts` · `src/integrator/controller/reporte.controller.ts`

Filtrado de catálogos en memoria de la aplicación. `WhodrugVacstempService` implementa seis métodos de búsqueda que recuperan la totalidad de registros de `TC_WHODRUG_VACS_TEMP` con `repository.find()` sin cláusula `WHERE`, y aplican el filtro en memoria del proceso Node.js con `.filter()`. Dentro del mismo archivo (líneas 23–37), existe una implementación correcta con `createQueryBuilder` y `WHERE` parametrizado que delega el filtrado a PostgreSQL, pero fue comentada y sustituida por el patrón de carga total. Las cláusulas `WHERE` de los métodos `getVaccinesByActiveIngredient()` (línea 138) y `getVaccinesByActIngTranslation()` (línea 218) también están presentes pero comentadas.

Servicios duplicados y errores silenciosos en `CatalogoService`. `IntegratorModule` registra `DatoEsaviService` en las líneas 186 y 204, y `AntecedenteEmbarazoService` en las líneas 179 y 205; NestJS sobrescribe silenciosamente el proveedor sin emitir advertencia. En `catalogo.service.ts`: el método `delete(uuid)` (línea 31) retorna `Promise.resolve(undefined)` sin ejecutar operación alguna, respondiendo HTTP 200 sin efecto; el método `findOne(uuid)` (línea 47) lanza `throw Error('')` con mensaje vacío, impidiendo identificar el contexto del fallo en los logs; el bloque `finally` de `create()` (línea 27) registra "Patient has been created" en un servicio que gestiona catálogos, no pacientes, generando entradas de log engañosas.

Validación de entrada en el controlador. `ReporteController` recibe parámetros de fecha mediante `@Query() aefiQuery: any` (tipo no estructurado) y ejecuta la validación de formato, presencia y coherencia de rango directamente en el cuerpo del método (líneas 41–75), en lugar de usar un DTO con decoradores de `class-validator` junto al `ValidationPipe` global ya configurado en `main.ts`. Los errores de validación retornan **HTTP 200** con `{ msg: 'Error' }` en el cuerpo en lugar del código HTTP 400 que corresponde semánticamente. Cuatro endpoints del mismo controlador tienen sus decoradores `@ApiResponse` comentados (líneas 91–98, 156–164, 177–184 y 200–207), dejando la documentación Swagger incompleta.

Tabla consolidada de observaciones

ID	Área	Descripción	Severidad
OBS-01a	Datos	Catálogos en CSV y Excel; archivo WHODrug duplicado con contenido divergente; nombres de archivo con fecha acoplados al código	Media
OBS-01b	Rendimiento	<code>OnApplicationBootstrap</code> genera +26,000 queries en cada reinicio; rangos Excel hardcodeados sin alerta de desbordamiento	Alta

ID	Área	Descripción	Severidad
OBS-01c	Arquitectura	7 tablas <code>TC_</code> sin patrón unificado; provincias con doble fuente; <code>TC_GRUPO_ETARIO</code> truncada como transaccional; 4 controladores sin endpoints; 2 gestores de paquetes	Media
OBS-02a	Seguridad	<code>DELETE /seed/tr-tables</code> ejecuta TRUNCATE dinámico sobre tablas de pacientes sin autenticación; API Key en Swagger sin guard implementado	Crítica
OBS-02b	Seguridad	Credenciales de PostgreSQL y Keycloak en texto plano en <code>docker-compose.yaml</code> commiteado al repositorio	Crítica
OBS-02c	Seguridad	CORS permite todos los orígenes cuando <code>CORS_ORIGINS</code> está vacía; <code>.env.example</code> la define sin valor	Alta
OBS-03a	Rendimiento	<code>WhodrugVacstempService</code> : 6 métodos cargan tabla completa en memoria para filtrar; implementación correcta con <code>WHERE</code> está comentada en el mismo archivo	Alta
OBS-03b	Código	Servicios duplicados en <code>providers[]</code> ; <code>delete()</code> retorna 200 sin implementar; <code>throw Error('')</code> vacío; log dice "Patient" en servicio de catálogo	Media
OBS-03c	Código	Validación de fechas con tipo <code>any</code> en controlador; errores de validación retornan HTTP 200; <code>@ApiResponse</code> comentados en 4 endpoints	Media